
Reading the Conversation Service

A line-by-line guide to `conversation.py` and `conversation_report.py`

Written for: a developer new to this codebase, and to some of the Python in it

Assumes: you can read basic Python. Everything beyond that is explained.

Covers: 824 + 220 lines – every line of both files, in order

Project: FluencyOS · backend/app/services/

1. How to use this document

These two files are the heart of the Conversation feature. This document goes through them in order, in small blocks, with the real line numbers next to the real code. Nothing is paraphrased — the code you see here was read straight out of the files.

You do not have to read this front to back. Section 2 is a primer on the Python and database ideas the code assumes you already know. If a piece of syntax in the walkthrough looks like noise, it is explained there.

The order I would read the code in

Reading a file top to bottom is the worst way to understand it, because the small helpers come first and they mean nothing until you know who calls them. Read in this order instead:

#	Read this	Because
1	<code>start_session</code> (line 437)	The simplest complete story: check, generate, write.
2	<code>submit_user_turn</code> (line 597)	One turn of conversation, start to finish. The core of the feature.
3	<code>end_session</code> (line 663)	The longest function, but by now every helper it calls is familiar.
4	<code>conversation_report.py</code> in full	Only 220 lines, and it is nearly all pure calculation.
5	The helpers at the top of <code>conversation.py</code>	Now you know where each one is used.

A habit worth forming. Almost every comment in these two files explains *why*, not *what* — and most of them describe a real bug that the line above is there to prevent. When a line looks over-complicated, read the comment before assuming it could be simpler. Several of them could be simpler, and were, and broke.

2. What you need to know first

The code uses a fairly small set of ideas over and over. Learn these nine and almost nothing in either file will look unfamiliar.

2.1 Talking to the database (sqlite3)

SQLite is a database that lives in a single file. Python talks to it through three things you will see constantly:

Thing	What it is
<code>conn</code>	A <i>connection</i> — an open line to the database file. Every function here takes one as its first argument rather than making its own, so the whole HTTP request shares one connection.
<code>conn.execute(sql, params)</code>	Runs one SQL statement and hands back a cursor you can read rows from.
<code>.fetchone()</code> / <code>.fetchall()</code>	Take one row, or every row, from that cursor. <code>fetchone()</code> gives <code>None</code> if there were none — which is why you see so many <code>if row is None</code> checks.

The `?` placeholder. You will never see a value pasted into a SQL string here. Instead:

```
Never: f"SELECT * FROM t WHERE id = '{user_id}'"
Always: conn.execute("SELECT * FROM t WHERE id = ?", (user_id,))
```

The `?` is a hole the database fills in itself, keeping the value strictly a value. If it were pasted into the string, a user id containing SQL would *become* SQL — that is SQL injection. Note the comma in `(user_id,)`: that trailing comma is what makes it a one-item tuple rather than just brackets around a variable, and leaving it out is a genuinely common beginner error.

`sqlite3.Row`. This project configures rows so you can read columns by name — `row["word"]` — instead of by position. When you see a parameter typed `sqlite3.Row`, picture "one record from the database, read like a dictionary".

`conn.commit()`. SQLite holds your writes in a transaction until you commit. Until then, nothing is permanent and other writers may be blocked. That is why you will see commits placed deliberately, and why one comment in `start_session` is specifically about not holding a transaction open across a slow AI call.

2.2 Type hints

The part after a colon or an arrow is a *hint*. Python does not enforce it at runtime; it is documentation that editors and checkers can verify.

```
def get_turns(conn: sqlite3.Connection, session_id: str) -> list[sqlite3.Row]:
```

Reads as: takes a connection and a string, gives back a list of rows. Two forms worth knowing:

- `str | None` — "a string, or nothing". The `|` means "or".

- `list[tuple[str, str]]` — a list of pairs of strings. This exact type is the conversation history:
`[("user", "hello"), ("assistant", "hi there")]`.

2.3 f-strings

A string prefixed with `f` can contain `{expressions}` that are evaluated and inserted. `f"cloud:{target['model']}"` becomes `"cloud:gpt-4"`. Used here mostly to build labels and, carefully, to build the `?,?,?` part of a SQL statement — never the values themselves.

2.4 Comprehensions and generator expressions

A compact way to build a list from another list.

```
user_turns = [t for t in turns if t["speaker"] == "user"]
```

"For each `t` in `turns`, keep it if the speaker is the user." The same shape inside round brackets is a *generator* — it produces items one at a time instead of building a whole list, which is what `sum(...)` and `any(...)` want:

```
fillers = sum(1 for w in all_words if w in FILLER_WORDS)
```

"Produce a 1 for every filler word, and add them up" — i.e. count them.

2.5 `next(..., None)` — first match or nothing

```
evidence_turn = next((t["turn_index"] for t in user_turns if ...), None)
```

Take the *first* item the generator produces and stop looking; if it produces none at all, give back `None`. Without that second argument it would raise an error instead. This is the idiom for "find the first thing matching, or nothing".

2.6 The `*` in a function signature

```
def start_session(conn, *, user_id: str, scenario: str, channel: str):
```

Everything after the bare `*` must be passed *by name*: `start_session(conn, user_id=..., scenario=...)`. You cannot pass them positionally. This is deliberate — `scenario` and `channel` are both strings, and positionally there would be nothing to stop you swapping them.

2.7 The leading underscore

`_generate_reply`, `_audio_dir`, `_words`. A leading underscore is a convention meaning *internal to this file* — not part of what other modules should call. Python does not enforce it. In these two files the distinction is real and useful: the names without underscores are the feature's public surface, and there are only about fifteen of them.

2.8 Pydantic — models that validate themselves

A `BaseModel` is a class that describes a shape of data and checks incoming data against it.

```
class ReportError(BaseModel):  
    bad: str  
    good: str  
    why: str
```

Hand it a dictionary through `model_validate(...)` and it either gives you a proper object or raises `ValidationError`. This matters enormously here, because the data being validated comes from a language model, which is free to return anything at all. `Field(default=..., description=...)` attaches both a fallback value and a human description to a field — and §5.4 shows the clever thing this file does with those descriptions.

2.9 `@dataclass(frozen=True)`

A *decorator* (the `@` line) modifies the class below it. `@dataclass` writes the boilerplate constructor for you from the field list. `frozen=True` makes instances immutable — once a `TranscriptMetrics` is built, nobody can quietly change a number inside it. For a value that represents a measurement, that is exactly right.

One more, and it is the most important idea in either file: `None` is not zero. Throughout this code, `None` is used to mean "this could not be measured", kept strictly apart from `0` meaning "measured, and it was zero". A text conversation has *no* speaking speed — reporting 0 words per minute would be a lie about the learner. Watch for this distinction; it drives several branches you would otherwise find puzzling.

3. The shape of the two files

Before the line-by-line, here is the shape of the thing. The Conversation feature lets a learner hold a spoken or typed conversation with an AI, which is told to work the learner's own vocabulary words into what it says. Afterwards the conversation is analysed, and the results feed the flashcard scheduler.

`conversation.py` has **three public entry points** that matter. Everything else in the file exists to serve them.

start_session()

|

submit_user_turn()

|

end_session()

line 437

line 597

line 663

Learner opens a conversation.
Picks target words, asks the AI to greet them,
writes the session and turn 0.

Learner says or types something.
Transcribe → reshape history → generate reply
→ write both turns.
Called once per exchange, many times.

Learner leaves.
Ask the AI to judge, compute the numbers,
write the report, feed the scheduler.

`conversation_report.py` is the analysis half, and it is built around one rule stated in its own docstring:

Every field in the report is either DETERMINISTIC or JUDGED, and the two never mix. Deterministic means computed from the transcript by plain arithmetic — reproducible, free, testable. Judged means asked of the language model, because it genuinely needs understanding of language. A model asked for a "fluency score" will happily invent one; a model asked whether a learner used a word correctly is doing work only a language model can do.

Deterministic (computed here)	Judged (asked of the AI)
words per minute	grammatical precision
filler rate per 100 words	the 3 most instructive errors
average response delay	self-corrections
longest uninterrupted run	how each target word was used
type–token ratio, above-level words	the closing summary
pronunciation (from STT confidence)	

4. conversation.py, line by line

Thirty-seven blocks covering all 824 lines. Each shows the real code at its real line numbers, followed by what it does and why.

1. The module docstringconversation.py · lines 1-9

```
1 """Session/turn orchestration for the Conversation feature: real target-word
2 selection, real local LLM chat generation, real STT/TTS on the voice
3 channel, and real post-chat analytics + review_logs writes.
4
5 Target-word selection is an honest substitute for real SRS "due" scheduling
6 (which doesn't exist anywhere in this app yet, see 0006_vocabulary.sql):
7 words with no conversation usage yet are preferred, tie-broken by most
8 recently saved.
9 """
```

A string at the very top of a file is its *docstring* — the file explaining itself. This one tells you two things.

First, what the file is responsible for: sessions and turns, choosing target words, calling the LLM, calling speech-to-text and text-to-speech, and writing analytics.

Second, and more useful, an honest admission. Target-word selection is called "an honest substitute for real SRS due scheduling". **SRS** is spaced repetition — the idea that you should review a word right before you forget it. Proper SRS computes a due date per word. This file does something simpler: prefer words never used in conversation, break ties by most recently saved. The docstring says so plainly rather than letting you assume the sophisticated version is in there.

2. Imports — what this file depends onconversation.py · lines 11-33

```
11 import json
12 import os
13 import sqlite3
14 from pathlib import Path
15
16 from pydantic import ValidationError
17
18 from app.config import settings
19 from app.services import conversation_report, review
20 from app.services.voice import (
21     cloud_llm_engine,
22     engine_health,
23     engine_status,
24     gemini_llm_engine,
25     llm_chat_engine,
26     model_catalog,
27     model_manager,
28     stt_engine,
29     tts,
30 )
31 from app.services.voice.errors import EngineUnavailable
32 from app.utils.ids import uuid7
33 from app.utils.time import iso8601_utc_now
```

Read imports as a dependency list. This file leans on:

Import	What it gives
--------	---------------

<code>json</code>	Turning Python lists into text for storage and back. Target word ids and the report are both stored as JSON text in a single column.
<code>os</code>	Used once, for <code>os.replace</code> at line 562 — an atomic file move.
<code>sqlite3</code>	Only for type hints here; the actual connection is handed in.
<code>Path</code>	Filesystem paths as objects rather than strings.
<code>ValidationError</code>	Pydantic's "this data did not match the shape" error.
<code>conversation_report,</code> <code>review</code>	The analysis half, and the flashcard scheduler.
the <code>voice</code> block	Nine engine modules: the three LLM providers, speech-to-text, text-to-speech, and the catalog/manager/status/health helpers around them.
<code>uuid7, iso8601_utc_now</code>	Id and timestamp generation.

Why `uuid7` and not `uuid4`? A UUIDv7 has the creation time built into its front, so sorting ids sorts by creation order. Rows written later sort later, which keeps database indexes tidy and makes ids debuggable at a glance.

3. The two constants

`conversation.py` · lines 35–54

```

35 TARGET_WORD_COUNT = 8
36
37 SCENARIOS: dict[str, dict[str, str]] = {
38     "free": {
39         "label": "Free talk",
40         "brief": "An open, casual conversation about whatever comes up – the learner's week, opinions, small
updates.",
41     },
42     "coffee": {
43         "label": "Order coffee",
44         "brief": "You are a barista at a coffee shop. Stay in character; take the learner's order, ask natural
follow-ups (size, milk, name for the cup).",
45     },
46     "job": {
47         "label": "Job interview",
48         "brief": "You are interviewing the learner for a job they're applying to. Ask realistic interview
questions and react to their answers.",
49     },
50     "debate": {
51         "label": "Debate a topic",
52         "brief": "Pick a everyday debatable topic and take a clear side, pushing back on the learner's points
respectfully.",
53     },
54 }
```

`TARGET_WORD_COUNT = 8` — how many vocabulary words get injected into a session.

`SCENARIOS` is a dictionary of dictionaries. The outer key (`"free"` , `"coffee"`) is what the database stores and the API accepts; the inner `label` is what the learner sees; the inner `brief` is dropped straight into the AI's instructions.

Notice how the briefs are written — second person, present tense: *"You are a barista at a coffee shop."* That is not decoration. It is the prompt, and it is phrased as a role instruction because that is what steers the

model.

Adding a scenario means adding one entry here. Nothing else in the backend needs to change — `_system_prompt` at line 373 looks the scenario up by key.

4. Where audio files live

conversation.py · lines 57–60

```
57 def _audio_dir() -> Path:
58     d = Path(settings.db_path).resolve().parent / "conversation_audio"
59     d.mkdir(parents=True, exist_ok=True)
60     return d
```

Takes the database file's location, goes to its parent folder, and puts a `conversation_audio` folder beside it.

- `.resolve()` turns a possibly-relative path into a full absolute one.
- `.parent` is the containing folder.
- `mkdir(parents=True, exist_ok=True)` creates it, creating any missing parent folders, and does *not* complain if it already exists. That combination makes this function safe to call on every request.

Keeping audio next to the database means one folder is the user's whole FluencyOS data — back up that, back up everything.

5. Which local model to use

conversation.py · lines 63–77

```
63 def selected_llm_option(conn: sqlite3.Connection, user_id: str) -> model_catalog.LlmOption:
64     row = conn.execute("SELECT llm_model_id FROM user_settings WHERE user_id = ?", (user_id,)).fetchone()
65     explicit_key = row["llm_model_id"] if row else None
66     if explicit_key:
67         return model_catalog.llm_option(explicit_key)
68
69     # No explicit choice made yet in Settings – prefer whichever option is
70     # actually downloaded rather than blindly falling back to the catalog's
71     # fixed recommended default, which may not be the one the user has. This
72     # keeps "download a model" alone sufficient to unblock Conversation, with
73     # no separate "now go select it" step required.
74     for option in model_catalog.LLM_OPTIONS:
75         if model_catalog.llm_is_downloaded(option):
76             return option
77     return model_catalog.llm_option(None)
```

The learner may have picked a model in Settings. Line 64 looks that up; line 65 reads it out defensively — `row["llm_model_id"] if row else None` handles the case where the user has no settings row at all.

The interesting part is lines 69–77. If no explicit choice was made, it does **not** fall back to the catalog's recommended default. It loops over the available options and returns the first one that is actually *downloaded*.

Why that matters. Imagine the recommended default is model A, but the learner downloaded model B. Falling back to A would leave Conversation refusing to start, with a message saying to download a model — while a perfectly good model sits on disk. The comment states the goal exactly: downloading a model alone is enough to unblock Conversation, with no separate "now go select it" step.

6. Where a reply will actually come from

conversation.py · lines 80–109

```
80 def llm_target(conn: sqlite3.Connection, user_id: str) -> dict:
81     """Resolves what a real LLM call should actually hit right now: either a
82     local catalog option (llama.cpp, loaded via Launch AI) or a cloud model
```

```

83     over OpenRouter or Gemini (no local load step – a live API call every
84     time either way).
85
86     Shape: {"provider": "local", "option": LlmOption} or
87           {"provider": "openrouter" | "gemini", "model": str, "api_key": str | None}.
88     `llm_model_id` is intentionally NOT reused for a cloud model string – it
89     stays local-catalog-key-only, so a cloud model id can never be misread as
90     a local one (see 0010_openrouter_llm.sql). `api_provider` picks which
91     cloud service 'api' mode actually means (see 0011_gemini_llm.sql)."""
92     row = conn.execute(
93         "SELECT llm_mode, api_provider, openrouter_api_key, openrouter_model, gemini_api_key, gemini_model "
94         "FROM user_settings WHERE user_id = ?",
95         (user_id,),
96     ).fetchone()
97     if row is not None and row["llm_mode"] == "api":
98         if row["api_provider"] == "gemini":
99             return {
100                 "provider": "gemini",
101                 "model": row["gemini_model"] or gemini_llm_engine.DEFAULT_MODEL,
102                 "api_key": row["gemini_api_key"],
103             }
104         return {
105             "provider": "openrouter",
106             "model": row["openrouter_model"] or cloud_llm_engine.DEFAULT_MODEL,
107             "api_key": row["openrouter_api_key"],
108         }
109     return {"provider": "local", "option": selected_llm_option(conn, user_id)}

```

This function answers one question: *if I asked for a reply right now, what would it hit?* It returns a small dictionary in one of two shapes, both documented in the docstring at lines 86–87.

```

{"provider": "local", "option": LlmOption}
{"provider": "openrouter" | "gemini", "model": str, "api_key": str | None}

```

Line 97: if `llm_mode` is `"api"`, the learner wants a cloud model. Line 98 then picks which cloud service. Note `row["gemini_model"] or gemini_llm_engine.DEFAULT_MODEL` on line 101 — in Python, `or` returns the first value that is "truthy", so this reads as "their choice, or the default if they have not made one". Empty string and `None` are both falsy, so this handles both.

Line 109 is the fallback: local.

The docstring's warning is worth reading twice. `llm_model_id` is deliberately *not* reused to store a cloud model name. It stays local-catalog-keys-only. If both kinds of name lived in one column, code reading it could not tell which it had, and a cloud model name could be misread as a local file to load. Two separate columns make that impossible rather than merely unlikely.

7. A human-readable engine name

conversation.py · lines 112–117

```

112 def llm_target_label(target: dict) -> str:
113     if target["provider"] == "openrouter":
114         return f"{target['model']}" (OpenRouter)
115     if target["provider"] == "gemini":
116         return f"{target['model']}" (Gemini)
117     return target["option"].label

```

Pure formatting, for the UI. Note that the two cloud branches read `target['model']` — a string — while the local branch reads `target["option"].label`, an attribute on an object. That difference is exactly the two shapes `llm_target` returns, and it is why every function here checks `provider` before reaching inside.

```

120 def _session_llm_target(conn: sqlite3.Connection, session: sqlite3.Row) -> dict:
121     """A conversation follows whatever engine Settings currently points at, so
122     switching model or provider takes effect on the chat already open instead
123     of only on the next one.
124
125     Sessions used to be pinned to the engine they started on, for consistent
126     voice across a transcript. In practice that traded a cosmetic consistency
127     for a real dead end: when the pinned engine became unusable (quota spent,
128     key revoked, model deleted) the conversation could never continue, and no
129     Settings change could rescue it. `model_id` now records what most recently
130     answered rather than dictating what must answer next."""
131     return llm_target(conn, session["user_id"])
132
133
134 def session_engine(conn: sqlite3.Connection, session: sqlite3.Row) -> dict:
135     """The engine this session's next turn will really call, named for the UI
136     so it can show the truth instead of inferring it from global state (which
137     is how it ended up claiming one engine while the turns called another)."""
138     target = _session_llm_target(conn, session)
139     return {"provider": target["provider"], "label": llm_target_label(target)}

```

`_session_llm_target` is a single line of code wrapped in a long comment, and the comment is the point.

Sessions *used to* be pinned to whatever engine they started on, so a transcript would keep one consistent voice. That sounds reasonable. The comment explains why it was abandoned: when the pinned engine became unusable — quota spent, key revoked, model deleted — the conversation could never continue, and *no Settings change could rescue it*. The learner was stuck with a dead conversation and no way out.

So the rule flipped. A conversation now follows whatever Settings currently points at. `model_id` in the database became a record of *what most recently answered* rather than a rule about *what must answer next*.

`session_engine` (134) exists so the UI can display the truth. Its comment notes the old bug: the UI inferred the engine from global state and ended up "claiming one engine while the turns called another".

A recurring lesson in this file. Storing a *decision* ("always use this") and storing a *fact* ("this is what happened") look identical in the database — one string column. They behave completely differently when something goes wrong.

9. Two tiny helpers

```

142 def _session_model_id(target: dict) -> str:
143     if target["provider"] == "gemini":
144         return f"cloud:gemini:{target['model']}"
145     if target["provider"] == "openrouter":
146         return f"cloud:{target['model']}"
147     return target["option"].key
148
149
150 def _cloud_engine(provider: str):
151     return gemini_llm_engine if provider == "gemini" else cloud_llm_engine

```

`_session_model_id` builds the string stored in the database. The prefixes matter: `cloud:gemini:gpt-x` versus `cloud:gpt-x` versus a bare local key. Anything reading the column can tell the three apart by prefix.

`_cloud_engine` picks a *module* — in Python, modules are ordinary values you can return from a function. The caller then calls `.generate_reply(...)` on whichever came back. This is how the file avoids `if provider == ...` branches at every call site.

10. Generating a reply — the dispatcher

conversation.py · lines 154–162

```
154 def _generate_reply(target: dict, system_prompt: str, history: list[tuple[str, str]], max_tokens: int = 120) ->
    str:
155     if target["provider"] != "local":
156         return _cloud_engine(target["provider"]).generate_reply(
157             system_prompt, history, api_key=target["api_key"], model=target["model"], max_tokens=max_tokens
158         )
159     option = target["option"]
160     return llm_chat_engine.generate_reply(
161         system_prompt, history, repo_id=option.repo_id, filename=option.filename, max_tokens=max_tokens
162     )
```

Nine lines that hide all three providers behind one call.

Line 155 checks for anything that is not local and routes it to the matching cloud module. Line 159 onwards handles local, which needs different arguments — `repo_id` and `filename` identify a model file on disk, where a cloud call needs `api_key` and `model`.

`max_tokens: int = 120` caps the reply length. A *token* is roughly three-quarters of a word, so 120 tokens is about two or three sentences — which matches the instruction the prompt gives the model at line 382.

Every provider module exposes the same two function names, `generate_reply` and `generate_json`. That shared shape is what makes this dispatcher three lines instead of thirty, and it is why adding a fourth provider would touch very little of this file.

11. Generating the analysis — and refusing to fail

conversation.py · lines 165–190

```
165 def _generate_analysis(
166     target: dict, transcript: list[tuple[str, str]], target_words: list[str]
167 ) -> conversation_report.LlmReportAnalysis:
168     """The judged half of the report. Prompt and parsing both come from
169     conversation_report, so the keys asked for are by construction the keys
170     read back — see that module's docstring for what went wrong when they
171     were maintained separately."""
172     system_prompt = conversation_report.report_system_prompt(target_words)
173     user_prompt = conversation_report.report_user_prompt(transcript)
174
175     if target["provider"] != "local":
176         raw = _cloud_engine(target["provider"]).generate_json(
177             system_prompt, user_prompt, api_key=target["api_key"], model=target["model"], max_tokens=700
178         )
179     else:
180         option = target["option"]
181         raw = llm_chat_engine.generate_json(
182             system_prompt, user_prompt, repo_id=option.repo_id, filename=option.filename, max_tokens=700
183         )
184     # A small model will sometimes omit a key or mistype a value; a partial
185     # analysis is still worth showing, so tolerate that rather than failing the
186     # whole report. What it cannot do is invent a field nobody asked for.
187     try:
188         return conversation_report.LlmReportAnalysis.model_validate(raw)
189     except ValidationError:
190         return conversation_report.LlmReportAnalysis()
```

The judged half of the report. Lines 172–173 get both prompts from `conversation_report` rather than writing them here — §5.4 explains why that indirection is load-bearing.

Lines 175–183 dispatch by provider exactly as `_generate_reply` does, but call `generate_json` and allow far more room (`max_tokens=700`) because a report is much longer than a conversational reply.

Lines 187–190 are the part worth studying:

```

try:
    return conversation_report.LlmReportAnalysis.model_validate(raw)
except ValidationError:
    return conversation_report.LlmReportAnalysis()

```

Try to fit what the model returned into the expected shape. If it does not fit, return an *empty* analysis instead of crashing. Because every field in `LlmReportAnalysis` has a default, an empty one is perfectly valid — the report simply shows nothing for the judged fields while all the computed ones still appear.

The comment draws a careful line. A small model will sometimes omit a key or mistype a value, and a partial analysis is still worth showing. What it cannot do is *invent a field nobody asked for* — Pydantic rejects unknown keys. Tolerant about what is missing, strict about what is extra.

12. Is everything downloaded?

conversation.py · lines 193–213

```

193 def _readiness_for_target(target: dict, channel: str, tts_name: str = tts.DEFAULT_ENGINE) -> dict:
194     if target["provider"] == "local":
195         llm_ready = model_catalog.llm_is_downloaded(target["option"])
196     else:
197         llm_ready = bool(target["api_key"])
198     if channel == "voice":
199         stt_ready = model_catalog.stt_is_downloaded()
200         # Only the *selected* voice engine has to be downloaded. Having
201         # Kokoro on disk says nothing about whether Pocket TTS is, and a
202         # session that would fail on the engine it will actually use should
203         # say so up front rather than at the first attempt to speak.
204         tts_ready = tts.is_downloaded(tts_name)
205     else:
206         stt_ready = tts_ready = True
207     return {
208         "ready": llm_ready and stt_ready and tts_ready,
209         "llm": llm_ready,
210         "stt": stt_ready,
211         "tts": tts_ready,
212         "llm_model_label": llm_target_label(target),
213     }

```

"Ready" means different things per provider, and this function encodes that.

Lines 194–197: local readiness is "is the model file on disk"; cloud readiness is "is there an API key". A cloud provider has nothing to download.

Lines 198–206: speech engines only matter on the voice channel. On a text channel, lines 206 sets both to `True` — `stt_ready = tts_ready = True` assigns both names at once. Text conversations genuinely do not need them.

Line 204 has a comment worth noting: only the *selected* TTS engine needs to be downloaded. Having Kokoro on disk says nothing about whether Pocket TTS is, and a session that would fail on the engine it is actually going to use should say so up front rather than at the first attempt to speak.

The returned dictionary reports both the overall verdict and each part, so the caller can say precisely *what* is missing.

13. Ready vs. launched, and current vs. pinned

conversation.py · lines 216–245

```

216 def readiness(conn: sqlite3.Connection, user_id: str, channel: str) -> dict:
217     """What a session needs before it can actually run – checked up front so
218     Conversation can refuse to start with a clear message instead of the old

```

```

219     behavior of silently kicking off a multi-minute implicit download (or
220     now, failing deep inside an HTTP request). For the cloud provider,
221     "ready" means a real API key is configured – there's no download step.
222     Reflects the user's *current* Settings preference – see
223     _ensure_ready_for_target for the session-pinned equivalent."""
224     return _readiness_for_target(llm_target(conn, user_id), channel, tts.selected_name(conn, user_id))
225
226
227 def _ensure_ready_for_target(target: dict, channel: str, tts_name: str = tts.DEFAULT_ENGINE) -> None:
228     r = _readiness_for_target(target, channel, tts_name)
229     if r["ready"]:
230         return
231     missing = [name for name, ok in (("LLM", r["llm"]), ("STT", r["stt"]), ("TTS", r["tts"])) if not ok]
232     raise EngineUnavailable(
233         f'{"/".join(missing)} model{'s' if len(missing) > 1 else ''} not downloaded yet – "
234         "download it in Settings before starting a conversation."
235     )
236
237
238 def _ensure_ready(conn: sqlite3.Connection, user_id: str, channel: str) -> None:
239     """Checks the user's *current* Settings preference – right for
240     start_session (no session exists yet). submit_user_turn/end_session use
241     _ensure_ready_for_target with the session's own pinned target instead,
242     so switching Settings mid-conversation (e.g. back to local with nothing
243     downloaded) can't block turns in a session that's still perfectly able
244     to run on what it was actually started with."""
245     _ensure_ready_for_target(llm_target(conn, user_id), channel, tts.selected_name(conn, user_id))

```

Three short functions, and the distinction between them is the single most confusing thing in the file. Here it is laid out:

Function	Asks	Used by
<code>readiness</code> (216)	Is it downloaded/configured? (reporting only)	The UI
<code>_ensure_ready</code> (238)	Same, but raises if not — using current Settings	<code>start_session</code>
<code>_ensure_ready_for_target</code> (227)	Same, but for a specific target	<code>submit_user_turn</code>
<code>_ensure_launched</code> (270)	Is it loaded into memory ?	All three entry points

Lines 231–235 build the error message. The list comprehension collects the names of whichever parts failed, then `'/'.join(missing)` joins them with slashes, and `{'s' if len(missing) > 1 else ''}` pluralises "model" only when needed. The result reads as "LLM/TTS models not downloaded yet" or "STT model not downloaded yet" — correct English in both cases.

Downloaded is not the same as loaded. A 2 GB model file can sit on disk while no part of it is in memory. Loading it takes tens of seconds. That is why these are two separate checks, and why the learner presses "Launch AI" explicitly rather than having the first sentence they speak silently trigger a minute of loading.

14. Engine status, made provider-aware

conversation.py · lines 248–267

```

248 def full_engine_status(conn: sqlite3.Connection, user_id: str) -> dict:
249     """Like engine_status.status(), but provider-aware: the cloud LLM has no
250     local "loaded into memory" concept, so it reads as ready the instant an
251     API key is configured, rather than through the local model-residency
252     check (which would otherwise always show not_loaded for it)."""
253     target = llm_target(conn, user_id)
254     if target["provider"] != "local":
255         st = engine_status.status(None)
256         # A saved key is not a working key. "ready" here means a real request

```

```

257     # has actually succeeded – established by Launch AI, and revoked again
258     # the moment one genuinely fails. Read from a record rather than probed
259     # here, so polling this endpoint never spends quota.
260     st["llm"] = (
261         "ready"
262         if target["api_key"] and engine_health.is_verified(target["provider"], target["model"])
263         else "not_loaded"
264     )
265     return st
266     option = target["option"]
267     return engine_status.status(str(model_manager.llm_model_path(option.repo_id, option.filename)))

```

The generic `engine_status.status()` checks whether a local model file is resident in memory. For a cloud provider that check makes no sense and would always report "not loaded".

Lines 260–264 substitute a different meaning for cloud: ready if there is a key *and* `engine_health.is_verified(...)` says a real request has actually succeeded.

"A saved key is not a working key." The comment is precise about the design. Verification is *recorded* when Launch AI spends one real request, and revoked when a request genuinely fails. It is read from that record here, never probed — because this endpoint gets polled, and probing on every poll would spend the learner's quota just to draw a status light.

15. The launch gate

conversation.py · lines 270–294

```

270 def _ensure_launched(needs: set[str], *, llm_target: dict, tts_name: str | None = None) -> None:
271     """Distinct from _ensure_ready: that checks a model is *downloaded*/
272     *configured*, this checks it's actually *loaded into memory* – a check
273     that only applies to the local provider, since a cloud API call has no
274     load step at all (once _ensure_ready confirmed a key exists, it's
275     already "launched"). For local, this is pinned to whichever model
276     'llm_target' says is actually selected right now – without that,
277     switching to a different (downloaded) model in Settings would still
278     read as "ready" off whatever was previously loaded, and the next turn
279     would silently pay a reload mid-conversation instead of the learner
280     choosing when via Launch AI."""
281     effective_needs = set(needs)
282     llm_path = None
283     if llm_target["provider"] != "local":
284         effective_needs.discard("llm")
285     else:
286         option = llm_target["option"]
287         llm_path = str(model_manager.llm_model_path(option.repo_id, option.filename))
288     st = engine_status.status(llm_path, tts_name)
289     missing = sorted(n.upper() for n in effective_needs if st.get(n) != "ready")
290     if missing:
291         raise EngineUnavailable(
292             f"AI isn't launched yet ({'/'.join(missing)} not loaded) – "
293             "use the Launch AI button at the top of the app, then try again."
294         )

```

The docstring opens by distinguishing itself from `_ensure_ready`: that checks *downloaded*, this checks *loaded into memory*.

Lines 281–287: start from what the caller says it needs. If the provider is not local, `discard("llm")` removes the LLM requirement entirely — a cloud API call has no load step, so once a key exists it is already "launched". Otherwise, work out the path of the model file that should be resident.

Line 289 collects the names that are not ready, uppercased and sorted so the message is stable rather than in whatever order a set happened to iterate.

Lines 291–294 raise an error that *tells the learner what to do*: use the Launch AI button at the top of the app. Compare that with a bare "engine not ready" and the difference in usefulness is obvious.

The subtle bug this prevents is in the docstring's last sentence. The check is pinned to whichever model `llm_target` says is selected *right now*. Without that, switching to a different downloaded model in Settings would still read as "ready" — because *something* was loaded — and the next turn would silently pay a full model reload mid-conversation. Pinning it means the learner chooses when that cost is paid.

16. Choosing the target words

conversation.py · lines 297–313

```
297 def _select_target_words(conn: sqlite3.Connection, user_id: str) -> list[sqlite3.Row]:
298     """An 'avoided' log means the word was offered as a target but the learner
299     never actually said it – counting that as practice would push exactly the
300     words that still need work to the back of the queue, so only outcomes
301     where the word was really produced count as usage here."""
302     return conn.execute(
303         """
304         SELECT vw.id, vw.word, vw.definition,
305                (SELECT COUNT(*) FROM review_logs rl
306                 WHERE rl.vocab_word_id = vw.id AND rl.outcome <> 'avoided') AS usage_count
307         FROM vocab_words vw
308         WHERE vw.user_id = ?
309         ORDER BY usage_count ASC, vw.created_at DESC
310         LIMIT ?
311         """,
312         (user_id, TARGET_WORD_COUNT),
313     ).fetchall()
```

One SQL statement, and the subquery on lines 305–306 is where the thinking is.

For each vocabulary word, count that word's rows in `review_logs` — *excluding* rows where the outcome was 'avoided'. Then order by that count ascending (least-practised first), breaking ties by most recently saved.

Why exclude 'avoided'. An "avoided" log means the word was offered as a target and the learner never said it. If that counted as usage, the word would look practised and sink down the queue — so the words that most need work would be pushed to the back precisely *because* the learner keeps failing to use them. The filter makes only genuine production count.

`LIMIT ?` takes `TARGET_WORD_COUNT`, passed as a parameter rather than pasted in — the same placeholder discipline applies to numbers.

17. Looking up specific words safely

conversation.py · lines 316–326

```
316 def _words_by_ids(conn: sqlite3.Connection, user_id: str, word_ids: list[str]) -> list[sqlite3.Row]:
317     """Scoped to the owner, so a seeded id can't pull in someone else's word.
318     Silently drops ids that no longer exist – a word deleted since the original
319     session shouldn't block repeating it."""
320     if not word_ids:
321         return []
322     placeholders = ",".join("?" for _ in word_ids)
323     return conn.execute(
324         f"SELECT id, word, definition FROM vocab_words WHERE user_id = ? AND id IN ({placeholders})",
325         [user_id, *word_ids],
326     ).fetchall()
```

This backs the "practise these again" feature, where the caller supplies word ids. Two safety properties, both in the docstring:

Scoped to the owner. The `WHERE user_id = ?` is what stops a supplied id from pulling in another user's word.

Silently drops missing ids. A word deleted since the original session should not block repeating it, so ids that no longer match simply return nothing.

Line 322 builds the `?,?,?` list dynamically — `','.join("? " for _ in word_ids)` makes one placeholder per id. This is the one legitimate use of an f-string in SQL here: what is being inserted is *placeholders*, generated by this code from a count. The values still travel separately on line 325. The underscore in `for _ in` is the conventional name for "a loop variable I do not need".

Line 320 returns early on an empty list — without it, `IN ()` would be invalid SQL.

18. normalise_for_chat — the most important helper here

conversation.py · lines 329–367

```
329 def normalise_for_chat(pairs: list[tuple[str, str]]) -> tuple[list[str], list[tuple[str, str]]]:
330     """Reshape a transcript into something every provider will accept.
331
332     Returns (assistant_opening_lines, history), where history begins with a
333     'user' message and strictly alternates user/assistant.
334
335     Two real failures make this necessary, both observed in production:
336
337     1. **A session opens with the AI speaking.** Gemma's chat template rejects
338     that outright – measured against gemma-3-1b, ``system + assistant +
339     user`` raises "Conversation roles must alternate
340     user/assistant/user/assistant/...", while ``system + user + assistant +
341     user`` is fine. So every reply after the AI greeting failed on any
342     Gemma model. Leading assistant turns are lifted out here and handed to
343     the caller to fold into the system prompt, which keeps their content
344     without breaking the alternation rule. Gemini requires the same shape.
345
346     2. **Consecutive turns from one speaker.** A failed generation used to
347     leave the learner's turn committed with no reply, so the next attempt
348     saw two user turns in a row and hit the same template error – the
349     error then masked whatever had actually gone wrong. Orphans are no
350     longer created (see submit_user_turn), but sessions already in that
351     state still have to work, so same-role runs are merged into one
352     message rather than dropped.
353     """
354     merged: list[tuple[str, str]] = []
355     for role, text in pairs:
356         clean = (text or "").strip()
357         if not clean:
358             continue
359         if merged and merged[-1][0] == role:
360             merged[-1] = (role, f"{merged[-1][1]} {clean}")
361         else:
362             merged.append((role, clean))
363
364     opening: list[str] = []
365     while merged and merged[0][0] == "assistant":
366         opening.append(merged.pop(0)[1])
367     return opening, merged
```

If you read one helper in this file, read this one. Its docstring documents **two real production failures**.

Failure 1: a session opens with the AI speaking. Every FluencyOS conversation starts with the AI greeting the learner. But Gemma's chat template requires messages to strictly alternate user, assistant, user, assistant — starting with user. The docstring records the measurement: against gemma-3-1b, `system + assistant + user` raises "Conversation roles must alternate", while `system + user + assistant + user` is fine.

So *every* reply after the AI greeting failed on any Gemma model. The conversation died immediately, every time.

Failure 2: two turns in a row from one speaker. A failed generation used to leave the learner's turn saved with no reply. The next attempt then saw two user turns in a row and hit the same template error — and that error masked whatever had actually gone wrong originally.

How it fixes both

Lines 354–362 handle the second failure. Walk the turns; skip anything empty (lines 356–358, where `(text or "").strip()` guards against both `None` and whitespace); and if this turn has the same role as the previous one, **merge** them into a single message rather than appending. Two user turns in a row become one user message containing both.

Lines 364–367 handle the first. While the list still *starts* with an assistant message, pop it off and collect its text. What comes back is a pair:

```
return opening, merged
```

The `opening` lines are handed to the caller, which folds them into the system prompt instead (see line 389) — so the model still knows what it said, without those messages breaking the alternation rule.

Note the choice to merge rather than drop. Orphaned turns are no longer created — `submit_user_turn` was restructured to prevent it — but sessions already in that state still exist in real users' databases. Fixing the cause did not remove the duty to keep old data working.

19. Building the AI's instructions

conversation.py · lines 370–390

```
370 def _system_prompt(
371     scenario: str, target_words: list[sqlite3.Row], opening: list[str] | None = None
372 ) -> str:
373     scenario_cfg = SCENARIOS[scenario]
374     words_block = (
375         "\n".join(f"- {w['word']}: {w['definition']} or 'no definition on file'" for w in target_words)
376         or "(the learner has no saved vocabulary yet – just have a natural conversation)"
377     )
378     return (
379         f"You are Juno, a friendly AI conversation partner helping someone practice English. "
380         f"Scenario: {scenario_cfg['brief']}\n\n"
381         f"The learner is trying to use these target words naturally in this conversation:\n{words_block}\n\n"
382         "Rules: never say a target word yourself before the learner does. Keep replies to ONE or TWO "
383         "short sentences – real spoken conversation, not an essay. Speaking them aloud takes longer "
384         "than generating them, so length is what the learner waits on. Ask a follow-up question to "
385         "keep the conversation going."
386         # Whatever the AI already said cannot stay in the message list without
387         # breaking alternation (see normalise_for_chat), so it is given back
388         # here instead – the model still knows how it opened.
389         + (f"\n\nYou have already said this to the learner: {' '.join(opening)}" if opening else "")
390     )
```

This assembles the system prompt — the instructions the model gets before the conversation, which it follows but which the learner never sees.

Lines 374–377 build the word list. The generator makes one `- word: definition` line each, and `w['definition'] or 'no definition on file'` covers words saved without one. The trailing `or` on line

376 catches the empty case: if the learner has no vocabulary at all, the whole block is replaced with an instruction to just have a natural conversation.

Lines 378–385 are the prompt proper. Three rules are given:

- **Never say a target word before the learner does.** Otherwise the AI hands them the answer and "spontaneous" use becomes impossible to measure.
- **One or two short sentences.** With the reason attached — speaking them aloud takes longer than generating them, so length is what the learner waits on.
- **Ask a follow-up question.** Keeps the conversation from stalling.

Line 389 is where `normalise_for_chat` 's `opening` comes back in: *"You have already said this to the learner: ..."*. The content survives; the alternation rule is respected.

20. Launch and unload

conversation.py · lines 393–434

```
393 def launch_engines(conn: sqlite3.Connection, user_id: str) -> dict:
394     """The "Launch AI" action: explicitly loads whatever's downloaded into
395     memory right now, instead of leaving that to happen (or be refused, per
396     _ensure_launched) at first real use. Cloud providers have nothing to load
397     for the LLM itself – just need a configured key – but voice's local
398     STT/TTS still get warmed up either way."""
399     target = llm_target(conn, user_id)
400     if target["provider"] == "local":
401         if not model_catalog.llm_is_downloaded(target["option"]):
402             raise EngineUnavailable("No AI model downloaded yet – download one in Settings before launching
AI.")
403         llm_chat_engine.warm_up(target["option"].repo_id, target["option"].filename)
404     elif not target["api_key"]:
405         label = "Gemini" if target["provider"] == "gemini" else "OpenRouter"
406         raise EngineUnavailable(f"No {label} API key configured – add one in Settings before launching AI.")
407     else:
408         # The cloud equivalent of loading a model: spend one small request to
409         # establish that the key really works, instead of assuming it from the
410         # fact that something is saved.
411         _cloud_engine(target["provider"]).verify(api_key=target["api_key"], model=target["model"])
412     if model_catalog.stt_is_downloaded():
413         stt_engine.warm_up()
414     tts_name = tts.selected_name(conn, user_id)
415     if tts.is_downloaded(tts_name):
416         # Whichever engine this user has picked – warming the other one would
417         # hold a few hundred MB for a voice that is never going to be called.
418         tts.engine_for(tts_name).warm_up()
419     return full_engine_status(conn, user_id)
420
421
422 def unload_engines(conn: sqlite3.Connection, user_id: str) -> dict:
423     """The opposite of launch_engines: hands the memory back.
424
425     Each engine's own unload() takes the same lock generation holds, so this
426     waits for any in-flight turn rather than pulling a model out from under
427     it. A cloud LLM has nothing resident to free – its key stays verified, so
428     the conversation can carry on while the local speech models are released."""
429     target = llm_target(conn, user_id)
430     if target["provider"] == "local":
431         llm_chat_engine.unload()
432     stt_engine.unload()
433     tts.unload_all()
434     return full_engine_status(conn, user_id)
```

`launch_engines` is the "Launch AI" button. It loads what is needed, deliberately and visibly, instead of letting it happen at first use.

Lines 400–403: for local, refuse if nothing is downloaded, then `warm_up` — load the model into memory now.

Lines 404–411: for cloud, refuse if there is no key; otherwise `verify`. The comment calls this "the cloud equivalent of loading a model" — spend one small request to establish that the key really works, rather than assuming it from the fact that something is saved.

Lines 412–418 warm the speech engines, and line 418's comment explains the narrowness: only the engine this user picked. Warming the other would hold a few hundred megabytes for a voice that will never be called.

`unload_engines` (422) hands the memory back. Its docstring notes something easy to miss: each engine's `unload()` takes the same lock that generation holds, so unloading *waits* for any in-flight turn rather than pulling a model out from under it. And a cloud LLM has nothing resident to free — its key stays verified, so the conversation can carry on while the local speech models are released.

21. start_session — checks and word selection

conversation.py · lines 437–458

```
437 def start_session(
438     conn: sqlite3.Connection,
439     *,
440     user_id: str,
441     scenario: str,
442     channel: str,
443     seed_word_ids: list[str] | None = None,
444 ) -> str:
445     """`seed_word_ids` backs "practise this again": rather than picking a fresh
446     set, the new session reuses the words from the one being repeated, so the
447     words that just went badly are the ones that come straight back."""
448     _ensure_ready(conn, user_id, channel)
449     target = llm_target(conn, user_id)
450     _ensure_launched(
451         {"llm"} | ({"tts"} if channel == "voice" else set()),
452         llm_target=target,
453         tts_name=tts.selected_name(conn, user_id),
454     )
455
456     target_words = _words_by_ids(conn, user_id, seed_word_ids) if seed_word_ids else []
457     if not target_words:
458         target_words = _select_target_words(conn, user_id)
```

Entry point 1 of 3. The whole function is: check, generate, write.

Line 448 checks everything is downloaded. Lines 450–454 check everything is loaded. Note what is required: the LLM always, plus TTS only `if channel == "voice"`. The `{"llm"} | (...)` is set union — `|` between two sets combines them. Speech-to-text is *not* required here, because opening a session only needs the AI to speak, not to listen.

Lines 456–458 pick the words. If the caller supplied `seed_word_ids` — the "practise this again" case described in the docstring — reuse exactly those. Otherwise choose fresh ones. Line 457's `if not target_words` also catches the case where seeding was requested but every id has since been deleted, falling back gracefully.

22. start_session — generate before writing

conversation.py · lines 459–469

```
459 # Generate the opening BEFORE writing anything, for two reasons. It keeps
460 # a failure from leaving a session row with no turns behind – one that
461 # would then sit in the learner's history forever as an empty
462 # conversation, and which "resume" could only ever reopen empty. And it
463 # means no write transaction is held open across a call that can take
464 # minutes on a cold model load, which would block every other writer.
```

```

465     opening = _generate_reply(
466         target,
467         _system_prompt(scenario, target_words),
468         [("user", "(The learner has just joined. Greet them and open the conversation.)")],
469     )

```

The comment on lines 459–464 gives two independent reasons for generating the greeting *before* writing anything, and both are worth understanding.

Reason one: no empty sessions. If the session row were written first and generation then failed, the learner's history would contain a conversation with no turns in it — forever. "Resume" could only ever reopen it empty.

Reason two: no long-held write transaction. SQLite allows one writer at a time. Holding a transaction open across an AI call — which can take *minutes* on a cold model load — would block every other writer in the app for that whole time.

Line 468 is the seed message. Since the history must start with a user message (§4.14), a synthetic one is supplied: "(The learner has just joined. Greet them and open the conversation.)". It is never stored and the learner never sees it — it exists only to give the model a legal shape to reply to.

23. start_session — the writes

conversation.py · lines 471–489

```

471     session_id = uuid7()
472     conn.execute(
473         """
474         INSERT INTO conversation_sessions (id, user_id, scenario, channel, target_word_ids, model_id,
475         started_at)
476         VALUES (?, ?, ?, ?, ?, ?, ?)
477         """,
478         (
479             session_id,
480             user_id,
481             scenario,
482             channel,
483             json.dumps([w["id"] for w in target_words]),
484             _session_model_id(target),
485             iso8601_utc_now(),
486         ),
487     )
488     conn.commit()
489     _insert_turn(conn, session_id, 0, "ai", opening, channel)
490     return session_id

```

Line 471 mints the id. Lines 472–486 insert the session.

Note line 482: `json.dumps([w["id"] for w in target_words])` — the list of word ids is serialised to a JSON string and stored in one column. That is a pragmatic choice over a separate join table. It costs you the ability to query "which sessions used this word" in SQL, and buys simplicity; the code reading it back always does `json.loads` first, which you will see at lines 638–640 and 667.

Line 487 commits, and line 488 writes turn 0 — the AI's greeting, with `turn_index` zero and speaker `"ai"`. Every conversation therefore starts with an assistant turn, which is precisely the condition `normalise_for_chat` exists to handle.

24. _insert_turn — and the audio decision

conversation.py · lines 492–528

```

492 def _insert_turn(
493     conn: sqlite3.Connection,
494     session_id: str,
495     turn_index: int,

```

```

496     speaker: str,
497     text: str,
498     channel: str,
499     stt_confidence: float | None = None,
500     speech_seconds: float | None = None,
501 ) -> sqlite3.Row:
502     turn_id = uuid7()
503     # Deliberately no synthesis here. Kokoro runs slower than real time on this
504     # hardware, so making the whole reply's audio before returning meant the
505     # learner waited ~30s to see any reply at all. Audio is produced per
506     # sentence, on demand, once the text is already on screen.
507     audio_path = None
508
509     conn.execute(
510         """
511         INSERT INTO conversation_turns
512             (id, session_id, turn_index, speaker, text, audio_path, stt_confidence, speech_seconds, created_at)
513         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
514         """,
515         (
516             turn_id,
517             session_id,
518             turn_index,
519             speaker,
520             text,
521             audio_path,
522             stt_confidence,
523             speech_seconds,
524             iso8601_utc_now(),
525         ),
526     )
527     conn.commit()
528     return conn.execute("SELECT * FROM conversation_turns WHERE id = ?", (turn_id,)).fetchone()

```

A straightforward insert, with one deliberate omission explained at lines 503–507.

audio_path = None, always. No speech is synthesised here. The comment records the measurement: Kokoro runs slower than real time on this hardware, so making the whole reply's audio before returning meant *the learner waited about 30 seconds to see any reply at all*. Audio is instead produced per sentence, on demand, once the text is already on screen.

This one decision is why `ensure_audio_chunk` (541) and the chunk-serving endpoint exist at all. Text appears immediately; audio catches up.

Line 528 re-reads the row it just wrote and returns it, so the caller gets the complete record including any database-side defaults, rather than a half-built object assembled in Python.

25. Why the engine name is in the filename

conversation.py · lines 531–538

```

531 def audio_chunk_path(turn_id: str, index: int, engine: str = tts.DEFAULT_ENGINE) -> Path:
532     """The engine is part of the filename because the audio is that engine's
533     voice. Without it, switching engines would keep serving whatever the
534     previous one had already cached for a turn, so the change would appear to
535     do nothing on every reply already on screen. It also leaves the two free
536     to split replies differently without one's chunk 0 being served as the
537     other's."""
538     return _audio_dir() / f"{turn_id}.{engine}.{index}.wav"

```

The file is named `{turn_id}.{engine}.{index}.wav`. The docstring explains why the engine is in there: the audio *is that engine's voice*.

Without it, switching TTS engines would keep serving whatever the previous engine had already cached — so the change would appear to do nothing on every reply already on screen. It also lets the two engines split

replies differently without one's chunk 0 being served as the other's.

26. ensure_audio_chunk — and a race condition

conversation.py · lines 541–565

```
541 def ensure_audio_chunk(turn_id: str, text: str, index: int, engine: str = tts.DEFAULT_ENGINE) -> Path:
542     """Synthesizes one sentence of a reply, cached on disk after the first
543     request. Called when that sentence is about to be played rather than while
544     the learner is still waiting to read the reply."""
545     name = tts.normalise(engine)
546     chunks = tts.engine_for(name).split_for_streaming(text)
547     if index < 0 or index >= len(chunks):
548         raise IndexError(f"chunk {index} out of range for {len(chunks)} chunks")
549     path = audio_chunk_path(turn_id, index, name)
550     if not path.exists():
551         audio = tts.engine_for(name).synthesize(chunks[index])
552         # Written to a private temp file and moved into place, never straight
553         # to 'path'. The player fetches chunk N+1 while N is still audible, so
554         # two requests for the same chunk really can overlap (a replay, a
555         # retry, a second window) – and a FileResponse reading a path that
556         # another thread is still writing serves a truncated WAV, which plays
557         # as a click or as silence. os.replace is atomic, so a reader sees
558         # either the whole file or no file.
559         tmp = path.with_suffix(f".{uuid7()}.part")
560         try:
561             tmp.write_bytes(audio)
562             os.replace(tmp, path)
563         finally:
564             tmp.unlink(missing_ok=True)
565     return path
```

Lines 545–548 split the reply the same way the synthesiser does and bounds-check the requested index. Line 550 is the cache: `if not path.exists()` — synthesise only if it is not already on disk.

Lines 553–564 are the careful part, and the comment lays out a real concurrency bug.

The race. The player fetches chunk N+1 while chunk N is still audible, so two requests for the same chunk really can overlap — a replay, a retry, a second window. If the file were written directly to `path`, a reader could open it *while another thread was still writing* and serve a truncated WAV, which plays as a click or as silence.

The fix: write to a uniquely-named temporary file, then `os.replace(tmp, path)`. That operation is **atomic** at the filesystem level — a reader sees either the complete old state (no file) or the complete new one, never a half-written file.

The `try/finally` guarantees the temp file is cleaned up even if writing raises. `unlink(missing_ok=True)` does not complain if the move already succeeded and the temp file is gone.

27. Chunk texts, and two plain lookups

conversation.py · lines 568–594

```
568 def audio_chunk_texts(turn: sqlite3.Row, channel: str, engine: str = tts.DEFAULT_ENGINE) -> list[str]:
569     """The text of each audio piece this turn produces, in order. Only the AI
570     speaks aloud, and only on the voice channel.
571
572     The client highlights words in time with the voice, which means it needs
573     the same split the synthesizer used – not a guess at it. Returning the
574     text rather than only a count is what makes the two agree."""
575     if channel != "voice" or turn["speaker"] != "ai":
576         return []
577     return tts.engine_for(engine).split_for_streaming(turn["text"])
578
579
580 def audio_chunk_count(turn: sqlite3.Row, channel: str, engine: str = tts.DEFAULT_ENGINE) -> int:
```

```

581     """How many audio pieces this turn can produce."""
582     return len(audio_chunk_texts(turn, channel, engine))
583
584
585 def get_session_row(conn: sqlite3.Connection, session_id: str, user_id: str) -> sqlite3.Row | None:
586     return conn.execute(
587         "SELECT * FROM conversation_sessions WHERE id = ? AND user_id = ?", (session_id, user_id)
588     ).fetchone()
589
590
591 def get_turns(conn: sqlite3.Connection, session_id: str) -> list[sqlite3.Row]:
592     return conn.execute(
593         "SELECT * FROM conversation_turns WHERE session_id = ? ORDER BY turn_index", (session_id,)
594     ).fetchall()

```

`audio_chunk_texts` returns the *text* of each audio piece, not just how many there are. The docstring says why: the client highlights words in time with the voice, so it needs the same split the synthesiser used — not a guess at it. Returning the text is what makes the two agree.

Line 575 returns an empty list for anything that does not speak aloud: only the AI speaks, and only on the voice channel.

`audio_chunk_count` (580) is simply the length of that. `get_session_row` (585) and `get_turns` (591) are plain lookups — note that `get_session_row` filters on `user_id` as well as `id`, so asking for someone else's session id returns nothing rather than their data.

28. submit_user_turn — setup and transcription

conversation.py · lines 597–621

```

597 def submit_user_turn(
598     conn: sqlite3.Connection,
599     session: sqlite3.Row,
600     *,
601     text: str | None,
602     audio_bytes: bytes | None,
603 ) -> tuple[sqlite3.Row, sqlite3.Row]:
604     channel = session["channel"]
605     target = _session_llm_target(conn, session)
606     _ensure_ready_for_target(target, channel, tts.selected_name(conn, session["user_id"]))
607     # Record what is actually answering now, so the session's engine label
608     # reflects reality after a mid-conversation switch.
609     model_id = _session_model_id(target)
610     if session["model_id"] != model_id:
611         conn.execute("UPDATE conversation_sessions SET model_id = ? WHERE id = ?", (model_id, session["id"]))
612         conn.commit()
613     _ensure_launched(
614         {"llm"} | ({ "stt" } if audio_bytes is not None else set()) | ({ "tts" } if channel == "voice" else set()),
615         llm_target=target,
616         tts_name=tts.selected_name(conn, session["user_id"]),
617     )
618     stt_confidence = None
619     speech_seconds = None
620     if audio_bytes is not None:
621         text, stt_confidence, speech_seconds = stt_engine.transcribe(audio_bytes)

```

Entry point 2 of 3, and the core of the feature. This runs once per exchange.

Lines 604–606 resolve the engine and check it is downloaded.

Lines 607–612 update the session's recorded engine if it has changed since the last turn — this is the "record what actually answered" behaviour from §4.8. Note it only writes when the value differs, avoiding a pointless write per turn.

Lines 613–617 check what is loaded, and the requirement set is built conditionally: the LLM always, plus STT *only if audio was supplied*, plus TTS *only on the voice channel*. A typed turn in a voice session needs no

speech-to-text.

Lines 620–621: if audio came in, transcribe it. Note the triple return — `text, stt_confidence, speech_seconds`. That third value, the real duration of recognised speech, is what makes an honest words-per-minute possible later (§5.9).

29. submit_user_turn — the empty-speech guard

conversation.py · lines 623–634

```
623     text = (text or "").strip()
624     if not text:
625         # Audio was provided (the router already rejects "neither text nor
626         # audio") but STT came back empty – silence, too short a clip, or
627         # unclear speech. Sending an empty turn to the LLM isn't meaningful
628         # for any provider, and Gemini specifically rejects it outright
629         # (its "ends with a model turn" error is really this in disguise:
630         # an empty user part gets dropped from validation, leaving the
631         # prior AI turn looking like the last one). Fail honestly instead.
632         raise ValueError("Didn't catch any speech there – try speaking again, a bit louder or closer to the
        mic.")
633     turns = get_turns(conn, session["id"])
634     next_index = turns[-1]["turn_index"] + 1 if turns else 0
```

Line 623 normalises whatever we have. `(text or "").strip()` handles `None` and whitespace in one expression.

Lines 624–632 refuse to continue with nothing. The comment is a good example of debugging written down:

Audio was provided but speech-to-text came back empty — silence, too short a clip, or unclear speech. Sending an empty turn to the LLM is meaningless for any provider, and **Gemini rejects it outright**. Its error says the conversation "ends with a model turn", which is that problem in disguise: the empty user part gets dropped during validation, leaving the previous AI turn looking like the last one.

Without this guard, a learner who coughed instead of speaking would get a baffling error about model turns. With it, they get: *"Didn't catch any speech there — try speaking again, a bit louder or closer to the mic."*

Line 634 computes the next turn index from the last existing one, with `else 0` for an empty session.

30. submit_user_turn — rebuilding the history

conversation.py · lines 636–645

```
636     target_words = conn.execute(
637         f"SELECT id, word, definition FROM vocab_words WHERE id IN "
638         f"({','.join('? ' for _ in json.loads(session['target_word_ids']))})",
639         json.loads(session["target_word_ids"]),
640     ).fetchall() if json.loads(session["target_word_ids"]) else []
641
642     pairs = [("assistant" if t["speaker"] == "ai" else "user", t["text"]) for t in turns]
643     pairs.append(("user", text))
644     opening, history = normalise_for_chat(pairs)
645     system_prompt = _system_prompt(session["scenario"], target_words, opening)
```

Lines 636–640 fetch the session's target words. The nested `json.loads(session['target_word_ids'])` appears three times in one expression — parsing the same JSON repeatedly. It works, and it is the least elegant passage in the file.

Line 642 converts database rows into the `(role, text)` pairs the LLM expects. The key translation is `"assistant" if t["speaker"] == "ai" else "user"`: the database says `"ai"`, but every LLM API says `"assistant"`. The two vocabularies meet here.

Line 643 appends what the learner just said. Line 644 runs the whole thing through `normalise_for_chat`, and line 645 folds any leading AI turns into the system prompt.

31. submit_user_turn — generate first, then write both

conversation.py · lines 647–660

```
647     # Generate BEFORE writing anything. A failure here used to leave the
648     # learner's turn committed with no reply, and the next attempt then saw
649     # two user turns in a row — which Gemma's template rejects, so the session
650     # was permanently stuck behind an alternation error that said nothing
651     # about the original failure. Nothing is persisted unless there is a
652     # reply to persist alongside it.
653     reply = _generate_reply(target, system_prompt, history)
654
655     user_turn = _insert_turn(
656         conn, session["id"], next_index, "user", text, channel, stt_confidence, speech_seconds
657     )
658     ai_turn = _insert_turn(conn, session["id"], next_index + 1, "ai", reply, channel)
659
660     return user_turn, ai_turn
```

Lines 647–652 explain the ordering, and this is the same lesson as `start_session` but with sharper consequences.

What used to happen. The learner's turn was written first. If generation then failed, the turn sat there with no reply. The next attempt saw two user turns in a row — which Gemma's template rejects — so the session was *permanently stuck* behind an alternation error that said nothing at all about the original failure.

Line 653 generates. Only then do lines 655–658 write, and they write **both** turns together: the learner's, then the AI's. Nothing is persisted unless there is a reply to persist alongside it.

If generation raises, the database is exactly as it was. The learner sees an error and can simply try again — which is the whole point.

The function returns both new rows so the caller can send both to the UI in one response.

32. end_session — gathering the evidence

conversation.py · lines 663–682

```
663 def end_session(conn: sqlite3.Connection, session: sqlite3.Row) -> dict:
664     target = _session_llm_target(conn, session)
665     _ensure_launched({"llm"}, llm_target=target)
666     turns = get_turns(conn, session["id"])
667     target_word_ids = json.loads(session["target_word_ids"])
668     target_words = (
669         conn.execute(
670             f"SELECT id, word FROM vocab_words WHERE id IN ({','.join('?' for _ in target_word_ids)}")
671             , target_word_ids,
672         ).fetchall()
673         if target_word_ids
674         else []
675     )
676
677     transcript = [(t["speaker"], t["text"]) for t in turns]
678     analysis = _generate_analysis(target, transcript, [w["word"] for w in target_words])
679
680     cefr_row = conn.execute("SELECT cefr_level FROM users WHERE id = ?", (session["user_id"],)).fetchone()
681     target_cefr = conversation_report.resolve_cefr(cefr_row["cefr_level"] if cefr_row else None)
682     metrics = conversation_report.compute_metrics(turns, target_cefr)
```

Entry point 3 of 3. The longest function in the file, but it is a straight sequence with no branching to speak of.

Line 665 requires only the LLM — the report needs judgement, not speech.

Lines 666–675 gather the turns and the target words. The `if target_word_ids else []` on lines 673–674 guards the same empty- `IN` problem as §4.16.

Line 677 flattens turns into `(speaker, text)` pairs and line 678 asks the model to judge them.

Lines 680–681 look up the learner's CEFR level — the A1-to-C2 scale of language proficiency — falling back to a default if placement was never done. `cefr_row["cefr_level"] if cefr_row else None` is the usual defensive read. Line 682 computes all the deterministic numbers.

33. end_session — checking the model's homework

conversation.py · lines 684–718

```
684 user_turns = [t for t in turns if t["speaker"] == "user"]
685 ai_turns = [t for t in turns if t["speaker"] == "ai"]
686 routing = []
687 outcomes_for_logs: list[tuple[str, str]] = []
688 for w in target_words:
689     # Whether the learner said the word at all is a fact about the
690     # transcript, not a judgement – so it is established here, not asked.
691     evidence_turn = next(
692         (t["turn_index"] for t in user_turns if conversation_report.says_word(t["text"], w["word"])),
693         None,
694     )
695     outcome = conversation_report.match_word_usage(analysis.word_usage, w["word"]) or "avoided"
696
697     if evidence_turn is not None and outcome == "avoided":
698         # "avoided" means "never used at all", and the transcript shows
699         # otherwise – a claim the evidence disproves, so it does not
700         # stand. Observed with gemma-3-1b marking every word avoided in a
701         # session where the learner had plainly used all three; the report
702         # then read 0/3 and the words were logged as unpractised, pushing
703         # them back to the front of the queue they had just earned their
704         # way out of.
705         #
706         # Only this one outcome is corrected. "incorrect" is a judgement
707         # about *how* the word was used and nothing here can second-guess
708         # it; "avoided" is the only verdict that is checkable.
709         said_first = any(
710             t["turn_index"] < evidence_turn and conversation_report.says_word(t["text"], w["word"])
711             for t in ai_turns
712         )
713         # The spec's own distinction: prompted means used only after the
714         # AI said it. That much is visible in the transcript.
715         outcome = "prompted" if said_first else "spontaneous"
716
717     routing.append({"word": w["word"], "outcome": outcome, "evidence_turn": evidence_turn})
718     outcomes_for_logs.append((w["id"], outcome))
```

This is the most interesting passage in either file. It takes the model's verdict on each target word and *overrules it* when the transcript proves it wrong.

Lines 691–694 establish a fact: did the learner actually say this word, and in which turn? The comment is exact — *whether the learner said the word at all is a fact about the transcript, not a judgement* — so it is established here rather than asked of the model.

Line 695 takes the model's verdict, defaulting to `"avoided"` if it said nothing about this word.

Lines 697–715 are the correction. If the transcript shows the word was said but the model claimed `"avoided"`, the claim does not stand.

The bug this fixes, from the comment: gemma-3-1b marked *every* word avoided in a session where the learner had plainly used all three. The report read 0/3, and the words were logged as unpractised —

pushing them back to the front of the queue they had just earned their way out of. The learner was punished for the model's mistake.

Lines 709–715 then work out which correction to apply, using the specification's own distinction: if the AI said the word *first*, the learner was "prompted"; otherwise "spontaneous". Both are visible in the transcript.

Only this one verdict is corrected, and the comment is careful to say why. "incorrect" is a judgement about *how* a word was used, and nothing here can second-guess that. "avoided" means "never used at all" — the only verdict that is *checkable* against the transcript. The code overrules the model exactly where it has evidence, and not one step further.

34. end_session — assembling the report

conversation.py · lines 720–744

```
720 correct_count = sum(1 for r in routing if r["outcome"] in ("spontaneous", "prompted"))
721 # None, not 0: a session with no target words didn't score zero accuracy,
722 # it had nothing to be accurate about.
723 contextual_accuracy_pct = round(100 * correct_count / len(routing)) if routing else None
724
725 report = {
726     "report_version": conversation_report.REPORT_VERSION,
727     "session_id": session["id"],
728     "contextual_accuracy_pct": contextual_accuracy_pct,
729     "grammatical_precision": analysis.grammatical_precision,
730     # Type-token ratio is 0-1; the dial needs 0-100.
731     "lexical_range": round(100 * metrics.type_token_ratio),
732     "pronunciation_score": metrics.pronunciation_score,
733     "words_per_minute": metrics.words_per_minute,
734     "filler_rate_per_100w": metrics.filler_rate_per_100w,
735     "avg_response_delay_seconds": metrics.avg_response_delay_seconds,
736     "longest_run_words": metrics.longest_run_words,
737     "type_token_ratio": metrics.type_token_ratio,
738     "above_level_words": metrics.above_level_words,
739     "self_corrections": analysis.self_corrections,
740     "turn_count": len(turns),
741     "routing": routing,
742     "errors": [e.model_dump() for e in analysis.errors[:3]],
743     "summary": analysis.summary or "No summary was generated.",
744 }
```

Lines 720–723 compute accuracy. Note line 723 once more:

```
contextual_accuracy_pct = round(100 * correct_count / len(routing)) if routing else None
```

The comment states the principle plainly: *None, not 0 — a session with no target words didn't score zero accuracy, it had nothing to be accurate about.*

Lines 725–744 build the report dictionary. Reading down it, you can see the two sources interleaved: `analysis.` fields came from the model; `metrics.` fields were computed. Line 731 converts type–token ratio from its natural 0–1 range to the 0–100 the UI dial expects.

Line 742 truncates errors to three with `analysis.errors[:3]` — a hard cap regardless of how many the model returned. Line 743 supplies fallback text if the summary came back empty, which is exactly what happens after the `ValidationError` path in §4.10.

Line 726 stamps the report with `REPORT_VERSION`, which is what lets the UI distinguish "not measured back then" from a genuine zero.

```

746 now = iso8601_utc_now()
747 conn.execute(
748     "UPDATE conversation_sessions SET report_json = ?, ended_at = ? WHERE id = ?",
749     (json.dumps(report), now, session["id"]),
750 )
751 # Ending recomputes over the WHOLE transcript so far, so a re-end
752 # supersedes this session's previous logs rather than adding a second set
753 # – appending would make one conversation count twice, both in a word's
754 # usage history and in _select_target_words' "not practised yet" ordering.
755 conn.execute("DELETE FROM review_logs WHERE session_id = ?", (session["id"],))
756 # Spec §6.3: this is what makes conversation count as study. Each outcome
757 # is rated on the same scale a flashcard would be and applied to the same
758 # FSRs card, so producing a word out loud reschedules it exactly as
759 # answering a flashcard would – the project's actual thesis, rather than
760 # a log nobody acts on.
761 review.apply_conversation_outcomes(
762     conn, session["user_id"], session["id"], outcomes_for_logs
763 )
764
765 return report

```

Lines 747–750 store the report as JSON on the session row.

Line 755 deletes this session's existing review logs before writing new ones, and the comment on lines 751–754 explains the subtlety: ending recomputes over the *whole* transcript, so a re-end must **supersede** the previous logs rather than add a second set. Appending would make one conversation count twice — both in a word's usage history and in `_select_target_words` ' "not practised yet" ordering.

Lines 761–763 are the payoff of the entire feature.

This is the project's actual thesis, in three lines of code. Each outcome is rated on the same scale a flashcard answer would be, and applied to the same FSRs card through the same code path. Saying a word out loud in conversation reschedules it *exactly* as answering a flashcard would. Without this call, the report would be a log nobody acts on.

The mapping is: spontaneous → Easy, prompted → Good, incorrect → Again, avoided → Hard. That translation lives in `review.py`.

```

768 def list_sessions(conn: sqlite3.Connection, user_id: str) -> list[sqlite3.Row]:
769     return conn.execute(
770         "SELECT * FROM conversation_sessions WHERE user_id = ? ORDER BY started_at DESC", (user_id,)
771     ).fetchall()
772
773
774 def delete_session(conn: sqlite3.Connection, user_id: str, session_id: str) -> bool:
775     """Deletes a session and its turns (DB cascade handles conversation_turns
776     and nulls out review_logs.session_id – the real usage history those rows
777     represent outlives the session record itself). Audio files aren't
778     covered by any DB cascade, so they're unlinked from disk explicitly
779     first."""
780     row = conn.execute(
781         "SELECT id FROM conversation_sessions WHERE id = ? AND user_id = ?", (session_id, user_id)
782     ).fetchone()
783     if row is None:
784         return False
785
786     for turn in conn.execute("SELECT id, audio_path FROM conversation_turns WHERE session_id = ?",
787                             (session_id,)):
788         if turn["audio_path"]:

```

```

788         Path(turn["audio_path"]).unlink(missing_ok=True)
789         # Per-sentence audio is written lazily and named off the turn id, so it
790         # isn't reachable through audio_path.
791         for chunk in _audio_dir().glob(f"{turn['id']}.*.wav"):
792             chunk.unlink(missing_ok=True)
793
794     conn.execute("DELETE FROM conversation_sessions WHERE id = ?", (session_id,))
795     return True

```

`list_sessions` is a plain query, newest first.

`delete_session` is more interesting. Lines 780–784 check the session exists *and belongs to this user*, returning `False` rather than raising — the caller turns that into a 404.

Lines 786–792 delete audio files, and the docstring explains the division of labour: the database cascade handles `conversation_turns` and nulls out `review_logs.session_id` — the real usage history those rows represent outlives the session record. But **no database cascade can delete a file**, so audio is unlinked explicitly.

Line 791 uses a glob rather than a stored path, and line 789's comment says why: per-sentence audio is written lazily and named off the turn id, so it is not reachable through `audio_path` — which, as we saw at line 507, is always `None`.

37. The two counting helpers

conversation.py · lines 798–823

```

798 def word_usage_counts(conn: sqlite3.Connection, vocab_word_id: str) -> dict[str, int]:
799     """Per-outcome CONVERSATION usage for one word — the visible half of
800     Dynamic SRS Routing, surfaced on the Vocabulary entry screen.
801
802     The source filter is load-bearing. review_logs carried only conversation
803     rows until 0015 gave flashcard reviews the same table (spec §5.5's
804     dual-source review), and without it this counted both — so the Vocabulary
805     page reported "conversation usage: again 1x" for a word that had only
806     ever been answered on a flashcard. "again" is not something anyone can do
807     in a conversation."""
808     rows = conn.execute(
809         "SELECT outcome, COUNT(*) AS n FROM review_logs "
810         "WHERE vocab_word_id = ? AND source = 'conversation' GROUP BY outcome",
811         (vocab_word_id,),
812     ).fetchall()
813     return {row["outcome"]: row["n"] for row in rows}
814
815
816 def flashcard_review_counts(conn: sqlite3.Connection, vocab_word_id: str) -> dict[str, int]:
817     """The other half: how this word has been answered on flashcards."""
818     rows = conn.execute(
819         "SELECT outcome, COUNT(*) AS n FROM review_logs "
820         "WHERE vocab_word_id = ? AND source = 'flashcard' GROUP BY outcome",
821         (vocab_word_id,),
822     ).fetchall()
823     return {row["outcome"]: row["n"] for row in rows}

```

Both count rows in `review_logs` grouped by outcome — one filtered to conversation, one to flashcards.

The docstring on lines 802–807 records a real bug caused by the missing filter. `review_logs` originally held only conversation rows. Migration 0015 gave flashcard reviews the same table. Without `source = 'conversation'`, this counted both — so the Vocabulary page reported *"conversation usage: again 1x"* for a word that had only ever been answered on a flashcard.

And as the comment dryly notes: **"again" is not something anyone can do in a conversation**. The outcome vocabularies of the two sources do not even overlap, which is what made the bug visible.

Line 813 turns the result rows into a plain dictionary like `{"spontaneous": 3, "avoided": 1}`.

5. conversation_report.py, line by line

Fifteen blocks covering all 220 lines. This file is smaller, mostly pure calculation, and repays careful reading — the reasoning in it is unusually explicit.

1. The docstring — read this one properly

conversation_report.py · lines 1–16

```
1 """Post-chat analysis: what the report measures, and who measures it.
2
3 Every field is either DETERMINISTIC (computed here from the transcript, so it
4 is reproducible, free, and unit-testable) or JUDGED (asked of the LLM, because
5 it genuinely needs language understanding). Keeping that line sharp is what
6 makes the report defensible: a model asked for a "fluency score" will invent
7 one, whereas a model asked whether a learner used a word correctly is doing
8 work only a language model can do.
9
10 This module is also the single source of truth for the judged half. The prompt
11 is rendered FROM `LlmReportAnalysis`, and `end_session` parses INTO it, so the
12 keys requested and the keys read cannot drift apart. They did drift once: the
13 prompt asked for three keys while the reader expected six, leaving four report
14 fields permanently zero on every real session while the tests passed. See
15 docs/conversation-report-design.md.
16 """
```

This docstring is the design document for the whole report, in fourteen lines. It makes two claims.

Claim one: every field is either DETERMINISTIC or JUDGED. Deterministic means computed here from the transcript — reproducible, free, unit-testable. Judged means asked of the LLM, because it genuinely needs language understanding.

The justification is the sharpest sentence in the codebase: *"a model asked for a 'fluency score' will invent one, whereas a model asked whether a learner used a word correctly is doing work only a language model can do."* That is the test for whether something belongs on the judged side. Can it be computed? Then compute it. Does it need understanding? Then ask — but only then.

Claim two: this module is the single source of truth for the judged half. The prompt is rendered *from* `LlmReportAnalysis`, and `end_session` parses *into* it. So the keys requested and the keys read cannot drift apart.

Lines 12–14 record what happened when they could: the prompt asked for three keys while the reader expected six, leaving **four report fields permanently zero on every real session while the tests passed**. The tests passed because they fed in well-formed data directly, never exercising the prompt. §5.4 shows the fix.

2. Imports

conversation_report.py · lines 18–23

```
18 import re
19 from dataclasses import dataclass
20
21 from pydantic import BaseModel, Field
22
23 from app.services import cefr_lexicon
```

Four things. `re` for regular expressions, `dataclass` for the metrics container, Pydantic for the judged model, and `cefr_lexicon` — the project's own module that knows which CEFR band a word belongs to and how to reduce a word to its base form.

Notice what is *absent*: no database, no engine modules, no filesystem. This module receives data and returns data. That is what makes it easy to test.

3. Constants, and a version number

conversation_report.py · lines 25–38

```
25 # Lexical hesitation markers. Deliberately a short, explicit list of items that
26 # are unambiguously fillers in English rather than a cleverer heuristic – the
27 # number is reported as a rate, and a wrong word in this list quietly skews it.
28 FILLER_WORDS = frozenset({"um", "uh", "erm", "ah", "hmm", "mmm", "like", "basically", "actually"})
29
30 # Used only when a learner has no CEFR level recorded yet (placement not done).
31 DEFAULT_CEFR = "B1"
32
33 # Bumped whenever the report's shape changes. Reports written by an older
34 # version are still readable (every added field has a default) and the UI uses
35 # this to tell "not measured back then" apart from a genuine zero.
36 REPORT_VERSION = 2
37
38 _WORD_RE = re.compile(r"[a-zA-Z']+")
```

`FILLER_WORDS` (28) is the hesitation-marker list. A `frozenset` is an unchangeable set — chosen because membership testing is fast and nobody can accidentally add to it at runtime.

The comment explains a restraint: this is deliberately a short, explicit list of words that are *unambiguously* fillers, rather than a cleverer heuristic — because the number is reported as a rate, and a wrong word in this list quietly skews it. ("like" and "actually" are already borderline; they have real meanings too.)

`DEFAULT_CEFR = "B1"` (31) is used only when a learner has never been placed.

`REPORT_VERSION = 2` (36) is bumped whenever the report's shape changes. The comment explains its purpose: old reports stay readable because every added field has a default, and the UI uses this number to tell "not measured back then" apart from a genuine zero.

`_WORD_RE` (38) is a regular expression matching runs of letters and apostrophes. `[a-zA-Z']+` means "one or more characters from this set", so "don't" stays one word while punctuation and digits are excluded. It is compiled once at import rather than on every call.

4. ReportError

conversation_report.py · lines 41–44

```
41 class ReportError(BaseModel):
42     bad: str
43     good: str
44     why: str
```

Three strings describing one mistake: what the learner said, the correction, and a short reason. Because it is a Pydantic model, an entry missing any of the three is rejected — a correction with no `good` value is not a correction.

5. LlmReportAnalysis — the judged half, defined once

conversation_report.py · lines 47–71

```
47 class LlmReportAnalysis(BaseModel):
48     """The judged half of the report – the only things the LLM is asked for.
49
50     Field names here become the JSON keys in the prompt (see
51     `report_system_prompt`), so renaming one changes both halves at once.
52     """
53
54     word_usage: dict[str, str] = Field(
55         default_factory=dict,
56         description='per target word: "spontaneous" | "prompted" | "incorrect" | "avoided"',
57     )
```

```

58     grammatical_precision: int = Field(
59         default=0, description="0-100, how grammatically accurate the learner's turns were"
60     )
61     errors: list[ReportError] = Field(
62         default_factory=list,
63         description="the 3 most instructive mistakes, each as "
64         '{"bad": what they said, "good": the correction, "why": a short reason}',
65     )
66     self_corrections: int = Field(
67         default=0, description="how many times the learner caught and repaired their own mistake"
68     )
69     summary: str = Field(
70         default="", description="2-3 encouraging sentences ending with one concrete focus for next time"
71     )

```

This class *is* the contract with the language model. Five fields, each with a default and a description.

The docstring states the mechanism: **field names here become the JSON keys in the prompt**, so renaming one changes both halves at once.

Field	Default	Why that default
<code>word_usage</code>	empty dict	No verdicts → every word falls through to "avoided" in <code>end_session</code> , then gets corrected against the transcript.
<code>grammatical_precision</code>	0	An integer 0–100.
<code>errors</code>	empty list	No mistakes found is a legitimate outcome.
<code>self_corrections</code>	0	A genuine count that can really be zero.
<code>summary</code>	empty string	Replaced with fallback text at <code>conversation.py:743</code> .

`default_factory=dict` rather than `default={}` is important Python. A plain `{}` as a default would be created *once* and shared by every instance — the classic mutable-default bug. A factory builds a fresh one each time.

The `description` texts are not comments. They are about to become the prompt.

6. report_system_prompt — the trick that keeps them in sync

conversation_report.py · lines 74–88

```

74 def report_system_prompt(target_words: list[str]) -> str:
75     """Built from LlmReportAnalysis's own fields, so the model is asked for
76     exactly what the reader parses – never a hand-maintained second copy."""
77     shape = ", ".join(
78         f'"{name}": {field.description}' for name, field in LlmReportAnalysis.model_fields.items()
79     )
80     words_list = ", ".join(target_words) if target_words else "(none)"
81     return (
82         "You are a strict, structured language-learning analyst. Given a conversation transcript and "
83         "the target vocabulary words the learner was meant to practise, respond with ONLY a JSON "
84         "object (no prose, no markdown fences) with exactly these keys: {" + shape + "}. "
85         "'spontaneous' = used correctly and unprompted. 'prompted' = used correctly only after the AI "
86         "'said or hinted the word. 'incorrect' = attempted but used wrongly. 'avoided' = never used at all. "
87         f"Judge only the learner's turns, never the AI's. Target words: {words_list}."
88     )

```

Here is the mechanism the module docstring promised. Lines 77–79:

```

shape = ", ".join(
    f'"{name}": {field.description}' for name, field in
    LlmReportAnalysis.model_fields.items()
)

```

`model_fields` is Pydantic handing you the class's own field list at runtime. This walks it and builds a string like:

```
"word_usage": per target word: "spontaneous" | ..., "grammatical_precision": 0-100, ...
```

which line 84 drops into the prompt as the required JSON shape.

Why this is the whole point. The prompt is *generated from* the class that parses the response. Add a field to `LlmReportAnalysis` and the prompt asks for it automatically. Rename one and both sides change together. The bug from the docstring — prompt asking three keys, reader expecting six — has been made **structurally impossible** rather than merely fixed.

Lines 85–87 define the four outcome words precisely, because they must mean the same thing to the model as they do to `end_session`'s correction logic. The final instruction — *"Judge only the learner's turns, never the AI's"* — prevents the model from scoring its own output.

7. report_user_prompt

conversation_report.py · lines 91–93

```

91 def report_user_prompt(transcript: list[tuple[str, str]]) -> str:
92     lines = "\n".join(f'{"learner" if speaker == "user" else "AI"}: {text}' for speaker, text in transcript)
93     return f"Transcript:\n{lines}"

```

Formats the transcript as labelled lines. Note the translation back: `'learner' if speaker == 'user' else 'AI'`. The database says `user / ai`, and the model is shown `learner / AI` — plain words that describe the roles rather than API jargon.

8. TranscriptMetrics — the deterministic half

conversation_report.py · lines 96–110

```

96 @dataclass(frozen=True)
97 class TranscriptMetrics:
98     """The deterministic half. `None` means "not measurable for this session",
99     which is deliberately distinct from zero – a text session has no speech
100     rate at all, and reporting one would be an artifact of how fast the model
101     replied rather than anything about the learner."""
102
103     words_per_minute: int | None
104     filler_rate_per_100w: float
105     avg_response_delay_seconds: float | None
106     longest_run_words: int
107     type_token_ratio: float
108     above_level_words: list[str]
109     pronunciation_score: int | None
110     total_words: int

```

A frozen dataclass holding the eight computed values.

The docstring defines the convention the entire report depends on:

None means "not measurable for this session", deliberately distinct from zero. A text session has no speech rate at all, and reporting one would be an artifact of how fast the *model* replied rather than anything

about the learner.

You can read which fields are optional straight off the type hints. Three carry `| None`: `words_per_minute`, `avg_response_delay_seconds` and `pronunciation_score`. The rest are always computable — a transcript always has *some* word count, even if zero.

9. `_words`, and the substring trap

conversation_report.py · lines 113–128

```
113 def _words(text: str) -> list[str]:
114     return _WORD_RE.findall(text.lower())
115
116
117 def says_word(text: str, word: str) -> bool:
118     """Whether 'text' actually contains 'word' as a word.
119
120     A plain substring test finds "cat" inside "category" and "art" inside
121     "start", which is enough to credit a learner with a target word they
122     never said – and to point the report's evidence link at the wrong turn.
123     Matched on the same normalised lemmas the other lexical metrics use, so
124     "goes" counts as "go" here exactly as it does there."""
125     wanted = cefr_lexicon.base_forms(word)
126     if not wanted:
127         return False
128     return any(wanted & cefr_lexicon.base_forms(w) for w in _words(text))
```

`_words` (113) lowercases the text and pulls out every word-like run. Two lines, used everywhere below.

`says_word` (117) looks trivial and is not. The docstring names the trap:

A plain substring test finds "cat" inside "category" and "art" inside "start" — enough to credit a learner with a target word they never said, and to point the report's evidence link at the wrong turn.

The fix is lines 125–128. `cefr_lexicon.base_forms(word)` reduces a word to its base forms — so "goes", "going" and "went" all reduce toward "go". Line 128 then asks whether *any* word in the text shares a base form with the target.

The `&` is set intersection — the overlap between two sets. `wanted & base_forms(w)` is non-empty exactly when they share a form, and a non-empty set is truthy.

The docstring's last sentence matters for consistency: this uses the *same* normalisation as the other lexical metrics, so "goes" counts as "go" here exactly as it does there. One notion of "same word" across the whole report.

10. `match_word_usage` — forgiving where it should be

conversation_report.py · lines 131–150

```
131 VALID_OUTCOMES = ("spontaneous", "prompted", "incorrect", "avoided")
132
133
134 def match_word_usage(word_usage: dict[str, str], word: str) -> str | None:
135     """The model's verdict for 'word', matched forgivingly.
136
137     The prompt lists the target words and asks for them back as JSON keys, and
138     a small model does not reliably echo them verbatim – it lowercases a
139     capitalised word, or answers about "wonders" when asked about "Wonder". An
140     exact dict lookup turns any of that into a silent miss that reads as
141     "avoided", which is indistinguishable from the learner never trying. Keys
142     are matched on shared base forms instead, the same way the transcript is
143     searched."""
144     wanted = cefr_lexicon.base_forms(word)
145     if not wanted:
```

```

146         return None
147     for key, outcome in word_usage.items():
148         if wanted & cefr_lexicon.base_forms(key) and outcome in VALID_OUTCOMES:
149             return outcome
150     return None

```

`VALID_OUTCOMES` (131) is the allowed vocabulary.

`match_word_usage` solves a problem created by asking a small model to echo words back as JSON keys: it **does not reliably do so verbatim**. It lowercases a capitalised word, or answers about `"wonders"` when asked about `"Wonder"`.

An exact dictionary lookup turns any of that into a silent miss that reads as `"avoided"` — *indistinguishable from the learner never trying*. The learner is then penalised for the model's spelling.

So lines 147–149 match on shared base forms instead, the same way the transcript is searched. Note line 148 also checks `outcome in VALID_OUTCOMES` — a key that matches but whose value is invented (say, `"excellent"`) is skipped rather than trusted.

Returning `None` for no match is what lets `end_session` line 695 apply its `or "avoided"` default and then check it against the transcript.

11. resolve_cefr

conversation_report.py · lines 153–158

```

153 def resolve_cefr(level: str | None) -> str:
154     """A learner who hasn't been placed yet still gets a usable comparison
155     band rather than an above-level count that silently reads zero."""
156     if level and cefr_lexicon.is_valid_band(level):
157         return level.upper()
158     return DEFAULT_CEFR

```

If the learner has a valid recorded level, use it uppercased. Otherwise fall back to B1.

The docstring explains the choice: a learner who has not been placed yet still gets a usable comparison band, *rather than an above-level count that silently reads zero*. With no band there is nothing to be "above", so the metric would report zero — which looks like a real measurement of a learner using no advanced vocabulary.

12. compute_metrics — words, fillers, and speaking rate

conversation_report.py · lines 161–175

```

161 def compute_metrics(turns: list, target_cefr: str) -> TranscriptMetrics:
162     """Pure function over turn rows. `turns` items need `speaker`, `text`,
163     `created_at`, and optionally `stt_confidence` / `speech_seconds`."""
164     user_turns = [t for t in turns if t["speaker"] == "user"]
165     all_words = [w for t in user_turns for w in _words(t["text"])]
166     total_words = len(all_words)
167
168     fillers = sum(1 for w in all_words if w in FILLER_WORDS)
169     filler_rate = round(100 * fillers / total_words, 1) if total_words else 0.0
170
171     # Real speech time, when we have it. Text sessions never do, and using
172     # wall-clock there would measure the model's latency, not the learner.
173     speech_seconds = sum(_speech_seconds(t) for t in user_turns)
174     words_per_minute = round(total_words / (speech_seconds / 60.0)) if speech_seconds > 0 and total_words else
175     None

```

Described in its docstring as a **pure function** over turn rows: same input, same output, no side effects, nothing external. That is why it is the easiest thing in the feature to test.

Line 164 keeps only the learner's turns — the AI is never measured. Line 165 is a *nested* comprehension: for every turn, for every word in that turn, collect the word. Read the two `for` clauses left to right like nested loops.

Lines 168–169 count fillers and convert to a rate per 100 words, guarding division by zero with `if total_words else 0.0`.

Lines 173–174 compute speaking rate, and line 171's comment is the justification for the whole `speech_seconds` column:

Real speech time, when we have it. Text sessions never do, and using wall-clock there would measure the model's latency, not the learner. Wall-clock includes the AI thinking and the AI talking — none of which is the learner speaking.

Line 174 requires *both* some speech time and some words before reporting a number; otherwise `None`.

13. compute_metrics — delay, run length, variety, pronunciation conversation_report.py · lines 176–192

```
176 # How long the learner took to start answering – the AI's own generation
177 # time sits outside this window, unlike the old all-turn-gaps average.
178 delays = []
179 for prev, cur in zip(turns, turns[1:]):
180     if prev["speaker"] == "ai" and cur["speaker"] == "user":
181         delays.append(_seconds_between(prev["created_at"], cur["created_at"]))
182 avg_response_delay = round(sum(delays) / len(delays), 1) if delays else None
183
184 longest_run = max((len(_words(t["text"])) for t in user_turns), default=0)
185
186 lemmas = {cefr_lexicon.normalise(w) for w in all_words}
187 type_token_ratio = round(len(lemmas) / total_words, 3) if total_words else 0.0
188
189 above_level = sorted({w for w in lemmas if cefr_lexicon.is_above_level(w, target_cefr)})
190
191 confidences = [t["stt_confidence"] for t in user_turns if t["stt_confidence"] is not None]
192 pronunciation = round(100 * sum(confidences) / len(confidences)) if confidences else None
```

Lines 179–181 find response delays using a neat idiom:

```
for prev, cur in zip(turns, turns[1:]):
```

`zip` pairs up two sequences. Pairing a list with itself offset by one gives you every *consecutive pair*. (1st, 2nd), (2nd, 3rd), and so on. Line 180 then keeps only AI → learner transitions.

The comment notes what this deliberately excludes: *the AI's own generation time sits outside this window, unlike the old all-turn-gaps average*. Measuring every gap would have made a slow model look like a hesitant learner.

Line 184 finds the longest single turn. `default=0` on `max` prevents an error on an empty sequence.

Lines 186–187 compute **type-token ratio** — distinct words divided by total words. Say 50 words using 40 different ones and you score 0.8; repeat one word 50 times and you score 0.02. It is a standard measure of lexical variety. Line 186 reduces to lemmas first, so `"go"` and `"goes"` count once, not twice.

Line 189 finds words above the learner's band — evidence of reaching beyond their level. `sorted({...})` makes the output stable between runs.

Lines 191–192 derive a pronunciation proxy from speech-to-text confidence: if the recogniser was confident, the speech was probably clear. Only turns that actually have a confidence value count, and with none the result is `None` — not zero.

14. compute_metrics — the return

conversation_report.py · lines 194–203

```
194     return TranscriptMetrics(  
195         words_per_minute=words_per_minute,  
196         filler_rate_per_100w=filler_rate,  
197         avg_response_delay_seconds=avg_response_delay,  
198         longest_run_words=longest_run,  
199         type_token_ratio=type_token_ratio,  
200         above_level_words=above_level,  
201         pronunciation_score=pronunciation,  
202         total_words=total_words,  
203     )
```

Every field named explicitly. With eight values of similar types, keyword arguments are what stop two of them being silently swapped — the same reasoning as the `*` in `start_session`.

15. Two defensive helpers

conversation_report.py · lines 206–219

```
206 def _speech_seconds(turn) -> float:  
207     try:  
208         value = turn["speech_seconds"]  
209     except (KeyError, IndexError):  
210         return 0.0  
211     return float(value) if value else 0.0  
212  
213  
214 def _seconds_between(start_iso: str, end_iso: str) -> float:  
215     from datetime import datetime  
216  
217     start = datetime.fromisoformat(start_iso.replace("Z", "+00:00"))  
218     end = datetime.fromisoformat(end_iso.replace("Z", "+00:00"))  
219     return (end - start).total_seconds()
```

`_speech_seconds` (206) is written for a specific hazard. A `sqlite3.Row` raises if you ask for a column that does not exist, and `speech_seconds` was added later by migration 0012. A row read from an older code path may not have it. The `try/except` catches both `KeyError` and `IndexError` and returns 0.0, so the metric degrades instead of crashing. Line 211 also treats `None` and 0 alike.

`_seconds_between` (214) parses two ISO timestamps and returns the gap in seconds. The `.replace("Z", "+00:00")` is a real compatibility quirk: timestamps are stored ending in `Z` (Zulu, meaning UTC), which older `fromisoformat` cannot parse, while `+00:00` means the same thing and works everywhere.

The import sits *inside* the function rather than at the top of the file. That is unusual, and here it is harmless — the module is imported at startup and this keeps a rarely-used dependency out of that path.

6. A working guideline

Having been through both files, here is what to carry forward — the patterns that repeat, and how to work on this code without breaking it.

6.1 Six patterns that repeat everywhere

Pattern	Where you saw it	What it protects
Generate before writing	<code>start_session</code> 459, <code>submit_user_turn</code> 647	A failed AI call leaves the database untouched, rather than half-written and unusable.
<code>None</code> $\neq$ 0	723, and all through <code>TranscriptMetrics</code>	"Could not measure" never masquerades as a real measurement of zero.
Check evidence, not claims	<code>end_session</code> 697–715	The model's verdict is overruled where — and only where — the transcript disproves it.
Generate the contract	<code>report_system_prompt</code> 77	The prompt and the parser cannot drift apart, because one is built from the other.
Match on base forms	<code>says_word</code> , <code>match_word_usage</code>	One consistent notion of "the same word" across the whole report.
Fail with instructions	232, 291, 632	Every error tells the learner what to actually do about it.

6.2 Where to make a given change

You want to...	Go to
Add a conversation scenario	<code>SCENARIOS</code> , line 37. Nothing else in the backend changes.
Change how the AI behaves	<code>_system_prompt</code> , line 378.
Add a computed report metric	<code>TranscriptMetrics</code> + <code>compute_metrics</code> , then add it to the report dict at line 725 and bump <code>REPORT_VERSION</code> .
Add a judged report field	<code>LlmReportAnalysis</code> only — the prompt updates itself. Then read it at line 725 and bump the version.
Change how target words are chosen	<code>_select_target_words</code> , line 302.
Add an LLM provider	A module with <code>generate_reply</code> + <code>generate_json</code> , then <code>llm_target</code> , <code>_cloud_engine</code> and <code>_session_model_id</code> .
Change how a word outcome affects scheduling	Not here — <code>review.apply_conversation_outcomes</code> .

Bump `REPORT_VERSION` whenever the report's shape changes. Reports already stored in users' databases are read back with the current code. The version is how the UI tells "this was written before we measured that" apart from "we measured it and it was zero". Forgetting to bump it makes old reports quietly lie.

6.3 Five ways to break this code

1. **Writing the user's turn before generating the reply.** You will recreate the orphaned-turn bug: a turn with no reply, then an alternation error on the next attempt that hides the real cause.
2. **Returning 0 where the code returns `None`.** The report will state, confidently, that a learner spoke at 0 words per minute.
3. **Writing the prompt's JSON keys by hand.** That is the exact bug the docstring documents — four fields silently zero on every real session, with the tests still green.
4. **Using `in` for word matching.** `"art" in "start"` is `True`. Use `says_word`.
5. **Appending review logs on re-end instead of replacing.** One conversation counts twice, and the affected words wrongly drop down the practice queue.

6.4 How to verify a change

The deterministic half is genuinely easy to test, by design — `compute_metrics` is a pure function, so you can hand it a list of fake turn dictionaries and assert on the numbers with no database and no model. The existing tests are in `backend/tests/test_conversation.py` and `test_conversation_report.py`.

But remember how the four-zero-fields bug survived. The tests passed because they fed well-formed data straight into the parser, never exercising the prompt that asked for it. A test that never crosses the boundary you changed will not notice you broke it. For anything touching the prompt, check the generated string itself.

6.5 If you read nothing else

The single idea holding these two files together is **the line between what can be computed and what must be judged**. Everything else — the `None`s, the transcript checks that overrule the model, the self-generating prompt, the base-form matching — exists to keep that line sharp and to stop the model's guesses leaking into the half of the report that is supposed to be fact.